Contexto del proyecto

Peguen este archivo completo al inicio de cada conversación nueva con la IA.

Actualícenlo cada vez que terminen o publiquen un módulo.

Proyecto

Nombre: Minder (Beta)

Descripción: Minder es una aplicación web orientada a la organización personal y al cuidado del usuario. Combina recordatorios programables, contactos de emergencia con acceso rápido a la línea 123 y un directorio de droguerías y hospitales con horarios, teléfonos y pedidos de domicilio de medicamentos, todo en una interfaz sencilla. Es un proyecto académico de Programación Orientada a Objetos, construido a partir de un prototipo en HTML.

No incluye:

- Asistente de voz (Beta).
- Modo sin conexión y sincronización de datos.
- Mapa interactivo con "las más cercanas" (se reemplaza por una lista con "abierta ahora" y un enlace a Google Maps).
- Avisos reales a los contactos de emergencia (SMS o notificación) ni alarmas con la aplicación cerrada.
- Panel de administrador y sugerencias o reportes (el administrador trabaja con el SQL Editor de Neon).

Repositorio: [pendiente: se crea en la semana 1 como repositorio privado `minder` en GitHub]

Aplicación publicada: [pendiente: URL de Render, se obtiene en la semana 1]

Nivel del equipo

Somos estudiantes principiantes. Sabemos fundamentos de programación en Python y Java, programación orientada a objetos (clases, encapsulamiento, herencia), listas y HTML básico. No sabemos bases de datos ni desarrollo web con backend. Nunca hemos publicado una aplicación. Trabajamos en Windows con Visual Studio Code.

Tiempo disponible: 4 semanas, 2 horas por semana por persona, 3 integrantes (24 horas en total).

Integrantes

Integrante	Usuario de GitHub	Responsable de

Herramientas elegidas

- **Frontend:** HTML + CSS + JavaScript básico (el prototipo del equipo), con `fetch()` para hablar con el backend. Las páginas las entrega el mismo servidor Java desde la carpeta `public/`.
- **Backend:** Java 21 + Javalin 6.x, con Maven (se usa el wrapper `mvnw.cmd`; solo Luis instala Maven). Librerías: driver JDBC de PostgreSQL y BCrypt para las contraseñas.

- **Base de datos:** PostgreSQL en Neon (plan gratuito, sin tarjeta). La clave va en variables de entorno (`DB_URL`, `DB_USER`, `DB_PASSWORD`), nunca en el código ni en GitHub.
- **Publicación:** Render (Web Service gratuito, tipo Docker, conectado a GitHub). Usa un `Dockerfile` de unas 8 líneas que no se instala ni se ejecuta localmente. El servicio se duerme tras 15 minutos sin visitas; abrir la página 5 minutos antes de exponer.
- **Editor:** Visual Studio Code (con la extensión "Extension Pack for Java").
- **Control de versiones:** Git + GitHub. Ciclo diario: `git pull` → trabajar → `git add .` → `git commit -m "mensaje"` → `git push`.

Forma de organizar el código

Clases, con una carpeta por función. La ficha del proyecto ya está en clases y la presentación pide mostrar herencia y encapsulamiento; con funciones sueltas eso no se vería.

- `modelo/`: las clases de la ficha (Persona, Usuario, ContactoEmergencia, Farmacia, Recordatorio, Pedido), con atributos privados, getters/setters y las reglas, como la validación del teléfono de 10 dígitos.
- `repositorio/`: una clase por tabla (por ejemplo `UsuarioRepositorio`). Solo estas clases escriben SQL.
- `App.java`: las rutas. Recibe la petición, llama a las clases y responde JSON.

Los métodos de la ficha se reparten así: los que guardan, editan o eliminan los hace el repositorio; las reglas (`verificarHorario()`, `posponer()`, `cancelar()`) viven en la clase.

Herencia: `Persona` (abstracta) → `Usuario` y `ContactoEmergencia`. Relaciones "TIENE UN": Usuario tiene Recordatorios y Contactos; Pedido tiene un Usuario y una Farmacia.

Datos que guarda el sistema

Persona no tiene tabla: es una clase abstracta que solo existe en Java. Usuario y ContactoEmergencia heredan de ella, pero cada una tiene su propia tabla.

Nombre	Campos	Se relaciona con
`usuarios`	id, nombre, documento (único), edad, telefono (10 dígitos), correo (único), contrasena_hash, latitud, longitud, esta_conectado, rol (`cliente`, `administrador` o `farmacia`), activo	Un usuario tiene muchos recordatorios, contactos y pedidos; una cuenta con rol `farmacia` puede administrar una farmacia
`contactos_emergencia`	id, usuario_id, nombre, documento (opcional), telefono, parentesco (opcional)	Pertenece a un usuario (`usuario_id`)
`recordatorios`	id, usuario_id, titulo, descripcion, fecha_hora, cumplido, creado_en	Pertenece a un usuario (`usuario_id`)
`farmacias`	id, nombre, tipo (`drogueria` u `hospital`), direccion, telefono, latitud, longitud, hora_apertura, hora_cierre, abierto_24h, acepta_domicilios, usuario_id (opcional)	Tiene muchos pedidos; `usuario_id` es la cuenta con rol `farmacia` que la administra
`pedidos`	id, usuario_id, farmacia_id, descripcion, direccion_entrega, estado (`enviado`, `confirmado`, `rechazado`, `entregado` o `cancelado`), tiempo_entrega_min (opcional), creado_en, actualizado_en	Pertenece a un usuario (`usuario_id`) y a una farmacia (`farmacia_id`)

Estructura de carpetas

Estructura prevista; se confirma en el Módulo 0 y se actualiza si cambia.

```
minder/
├── pom.xml
├── mvnw.cmd
├── .mvn/
├── Dockerfile
├── .gitignore
├── README.md
├── CONTEXTO.md
├── sql/
│   └── 01_farmacias.sql
└── src/main/
    ├── java/
    │   ├── App.java
    │   ├── modelo/
    │   │   ├── Persona.java
    │   │   ├── Usuario.java
    │   │   ├── ContactoEmergencia.java
    │   │   ├── Farmacia.java
    │   │   ├── Recordatorio.java
    │   │   └── Pedido.java
    │   └── repositorio/
    │       ├── UsuarioRepositorio.java
    │       ├── ContactoRepositorio.java
    │       ├── FarmaciaRepositorio.java
    │       ├── RecordatorioRepositorio.java
    │       └── PedidoRepositorio.java
    └── resources/
        └── public/
            ├── index.html
            ├── estilos.css
            ├── sesion.js
            ├── registro.html
            ├── login.html
            ├── farmacias.html
            ├── recordatorios.html
            ├── contactos.html
            └── pedidos.html
```

Módulos

#	Módulo	Estado	Publicado	Responsable
0	Página mínima publicada: "Hola" + conexión a la base de datos (semana 1)	Pendiente	No	Todos
1	Directorio de farmacias, solo lectura (semana 2)	Pendiente	No	Todos
2	Cuentas: registro, login y herencia Persona → Usuario (semanas 2 y 3)	Pendiente	No	Todos
3	Recordatorios (semana 3)	Pendiente	No	Todos
4	Contactos de emergencia y botón 123 (semanas 3 y 4)	Pendiente	No	Todos
5	Pedidos de domicilio (semana 4)	Pendiente	No	Todos

Estados posibles: Pendiente, En curso, Terminado.

Un módulo solo está terminado cuando funciona en la versión publicada.

Cada módulo sigue el patrón: tabla → clase → repositorio → ruta → pantalla. Las tareas de cada módulo están repartidas por persona en el plan. Después del Módulo 5 viene el cierre (semana 4): pruebas completas, README y ensayo de la exposición de la herencia.

Módulo actual

[en qué módulo están trabajando y en qué tarea van]

Problemas pendientes

- Ninguno por ahora.

Decisiones importantes

- 2026-10-20: Presupuesto real de 24 horas (3 personas × 2 h × 4 semanas); por eso cada módulo se hace en su versión mínima y se recortó lo de la sección "No incluye".
- 2026-10-20: Backend en Java con Javalin, sin Spring, porque es la opción más simple para quien ya sabe Java y arranca desde un `main` normal.
- 2026-10-20: Base de datos PostgreSQL en Neon y no SQLite, porque el disco gratuito de Render es efímero y borraría los datos. Tampoco se usa el Postgres de Render (expira a los 30 días) ni Supabase (se pausa tras 7 días sin uso).
- 2026-10-20: Publicación en Render con un `Dockerfile` corto, porque Render no ejecuta Java de forma nativa. No se instala ni se ejecuta Docker en los computadores.
- 2026-10-20: Código organizado en clases (`modelo/`, `repositorio/`, `App.java`) para poder mostrar herencia y encapsulamiento en la exposición.
- 2026-10-20: `Persona` no tiene tabla; `usuarios` y `contactos_emergencia` son tablas separadas que repiten las columnas comunes.
- 2026-10-20: Se guarda `contrasena_hash` (BCrypt) y nunca la contraseña real. El teléfono se valida con el setter de Java y con un `CHECK` en la base de datos.
- 2026-10-20: Una sola tabla `usuarios` con la columna `rol` (`cliente`, `administrador`, `farmacia`) para un único login; `activo` reemplaza activar y suspender cuenta.
- 2026-10-20: "Abierta ahora" no se guarda; se calcula con `verificarHorario()` usando la zona `America/Bogota`. Las fechas de recordatorios se guardan como `TIMESTAMPTZ`.
- 2026-10-20: `crear()` y `enviarSolicitud()` de Pedido se unen: el pedido nace en estado `enviado`.
- 2026-10-20: Todas las pruebas usan datos inventados; no se suben datos reales de personas.
- 2026-10-20: Si se atrasan, se recorta en este orden: panel de farmacia del Módulo 5, "posponer" y aviso en pantalla del Módulo 3, edición de contactos, y por último el Módulo 5 completo.